

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практические задания
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI_KR5/FAPI_KR5.md

Контрольная работа №5 🍍

🐱 Как сдавать работу

1. Выполните задания в отдельном репозитории.
2. Загрузите исходный код в репозиторий (`commit + push`).
3. В **СДО** прикрепите отчёт в виде **ссылки на репозиторий с решением**.
4. В корне репозитория должен быть файл `README.md` с инструкцией по запуску проекта и тестов.

Рекомендуется, чтобы репозиторий содержал понятную структуру проекта: `app/` , `tests/` , `Dockerfile` , `docker-compose.yml` , `requirements.txt` или `pyproject.toml` .

📁 Общие требования

1. Проект должен запускаться локально через `uvicorn` .
2. Проект должен запускаться в Docker-контейнере.
3. Для HTTP-эндпоинтов используйте FastAPI и Pydantic-модели.
4. Для проверки работы API добавьте интеграционные тесты на `pytest` и `TestClient` .
5. Для WebSocket-части используйте `WebSocket` , `WebSocketDisconnect` и тестирование через `websocket_connect()` .

6. Код должен быть разделён минимум на несколько файлов: основной файл приложения, роутеры, схемы, зависимости, тесты.
 7. В ответах API должны использоваться корректные HTTP-статусы: 200 , 201 , 204 , 400 , 401 , 403 , 404 , 422 .
-

🌟 Задание 1. Интеграционные тесты для API задач

Ваша задача — создать FastAPI-приложение для управления задачами и покрыть его интеграционными тестами.

Функциональные требования

Создайте REST API для работы с задачами пользователя.

Каждая задача должна иметь поля:

- `id ( int )`: уникальный идентификатор задачи.
- `title ( str )`: название задачи, обязательно, от 3 до 80 символов.
- `description ( str | None )`: описание задачи, необязательно.
- `status ( str )`: статус задачи, одно из значений: `todo` , `in_progress` , `done` .
- `priority ( int )`: приоритет от 1 до 5.
- `owner_id ( int )`: идентификатор пользователя-владельца.

Для имитации авторизации используйте заголовок:

```
X-User-Id: 10
```

Если заголовок отсутствует или его невозможно привести к `int` , API должно возвращать ошибку 401 .

Необходимые маршруты

1. Создание задачи:

- Метод: `POST`
- Маршрут: `/tasks`
- Статус успешного ответа: 201

Пример запроса:

```
{
  "title": "Подготовить тесты",
  "description": "Написать интеграционные тесты для основных сценариев",
  "status": "todo",
  "priority": 4
}
```

Пример ответа:

```
{
  "id": 1,
  "title": "Подготовить тесты",
  "description": "Написать интеграционные тесты для основных сценариев",
  "status": "todo",
  "priority": 4,
  "owner_id": 10
}
```

2. Получение списка задач текущего пользователя:

- Метод: GET
- Маршрут: /tasks
- Параметры запроса:
 - status (str | None): фильтр по статусу.
 - min_priority (int | None): минимальный приоритет.

3. Получение одной задачи:

- Метод: GET
- Маршрут: /tasks/{task_id}
- Если задача не найдена или принадлежит другому пользователю, вернуть 404 .

4. Изменение статуса задачи:

- Метод: PATCH
- Маршрут: /tasks/{task_id}/status

Пример запроса:

```
{
  "status": "done"
}
```

5. Удаление задачи:

- Метод: `DELETE`
- Маршрут: `/tasks/{task_id}`
- Статус успешного ответа: `204`.

Требования к тестам

Создайте папку `tests/` и файл `tests/test_tasks.py`.

Проверьте минимум следующие сценарии:

1. Успешное создание задачи.
2. Ошибка `422`, если `title` короче 3 символов.
3. Ошибка `401`, если нет заголовка `X-User-Id`.
4. Пользователь видит только свои задачи.
5. Фильтрация задач по `status` и `min_priority`.
6. Успешное изменение статуса задачи.
7. Ошибка `404` при обращении к чужой или несуществующей задаче.
8. Успешное удаление задачи.

Подсказка: используйте фикстуру `client`, которая создаёт `TestClient(app)`, и фикстуру для очистки тестового хранилища перед каждым тестом.

🌟 Задание 2. Docker-контейнеризация FastAPI-приложения

Ваша задача — упаковать FastAPI-приложение из задания 1 в Docker-контейнер и подготовить удобный запуск через `docker compose`.

Требования к файлам

В корне проекта должны быть файлы:

```
Dockerfile
docker-compose.yml
.dockerignore
requirements.txt
README.md
```

Требования к Dockerfile

1. Используйте базовый образ `python:3.12-slim`.

2. Установите рабочую директорию `/app` .
3. Сначала копируйте файл зависимостей, затем устанавливайте зависимости.
4. После установки зависимостей копируйте исходный код приложения.
5. Запускайте приложение командой:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

6. Добавьте инструкцию `EXPOSE 8000` .
7. Не копируйте в образ `.venv` , `.git` , `__pycache__` , `.pytest_cache` .

Требования к `docker-compose.yml`

Создайте сервис `api` , который:

- собирается из текущей директории;
- пробрасывает порт `8000:8000` ;
- передаёт переменную окружения `APP_ENV=docker` ;
- перезапускается при ошибке с политикой `unless-stopped` .

Пример проверки после запуска:

```
docker compose up --build
curl http://localhost:8000/tasks -H "X-User-Id: 10"
```

Ожидаемый результат для пустого списка задач:

```
[]
```

Дополнительные требования

1. В `README.md` опишите команды:

```
python -m venv .venv
pip install -r requirements.txt
uvicorn app.main:app --reload
pytest
docker compose up --build
```

2. Добавьте маршрут проверки состояния приложения:

- Метод: `GET`
- Маршрут: `/health`

Ответ:

```
{
  "status": "ok",
  "env": "docker"
}
```

3. Напишите интеграционный тест, который проверяет маршрут `/health`.

🌟 Задание 3. WebSocket-комнаты для обмена сообщениями

Ваша задача — реализовать WebSocket-чат с комнатами и простым HTTP-интерфейсом для просмотра активных подключений.

Функциональные требования

Создайте WebSocket-маршрут:

```
/ws/rooms/{room_id}?username=alice
```

Где:

- `room_id (str)`: название комнаты.
- `username (str)`: имя подключившегося пользователя.

Если параметр `username` не передан или состоит только из пробелов, соединение нужно закрыть с кодом `1008`.

Логика работы

1. При подключении пользователя сервер должен добавить его в комнату.
2. Всем пользователям комнаты нужно отправить событие о подключении.
3. Пользователь может отправить JSON-сообщение следующего вида:

```
{
  "type": "message",
  "text": "Всем привет"
}
```

4. Сервер должен разослать сообщение всем клиентам этой комнаты в формате:

```
{
  "type": "message",
  "room_id": "python",

```

```
"username": "alice",
"text": "Всем привет"
}
```

5. Если сообщение длиннее 300 символов, сервер должен ответить отправителю:

```
{
  "type": "error",
  "detail": "Message is too long"
}
```

6. При отключении пользователя сервер должен удалить его из комнаты.

HTTP-маршрут для проверки комнат

Создайте маршрут:

- Метод: GET
- Маршрут: `/rooms/{room_id}/users`

Ответ должен возвращать список активных пользователей комнаты.

Пример ответа:

```
{
  "room_id": "python",
  "users": ["alice", "bob"]
}
```

Требования к реализации

1. Создайте класс `RoomManager`.
2. Внутри класса храните структуру данных для комнат и подключений.
3. Реализуйте методы:
 - `connect(room_id, username, websocket)`;
 - `disconnect(room_id, username, websocket)`;
 - `broadcast(room_id, payload)`;
 - `get_users(room_id)`.
4. Используйте `send_json()` и `receive_json()`.
5. Обработайте `WebSocketDisconnect`.

Требования к тестам

Создайте файл `tests/test_websocket.py`.

Проверьте минимум следующие сценарии:

1. Подключение к комнате с корректным `username`.
2. Отправка сообщения и получение ответа через WebSocket.
3. Два клиента в одной комнате получают одно и то же сообщение.
4. Пользователи из разных комнат не получают чужие сообщения.
5. Слишком длинное сообщение возвращает событие `error`.
6. После отключения пользователя маршрут `/rooms/{room_id}/users` не возвращает его в списке.

Подсказка: для тестов используйте `client.websocket_connect("/ws/rooms/python?username=alice")`.

🌟 Задание 4. Внедрение зависимостей и расширенная маршрутизация

Ваша задача — перестроить приложение в модульную архитектуру с использованием `APIRouter`, зависимостей и проверки прав доступа.

Структура проекта

Реализуйте следующую структуру:

```
app/  
  __init__.py  
  main.py  
  dependencies.py  
  schemas.py  
  storage.py  
  routers/  
    __init__.py  
    tasks.py  
    users.py  
    admin.py
```

Требования к маршрутизаторам

1. `app/routers/tasks.py`

Префикс: `/tasks`

Маршруты:

- `POST /tasks`
- `GET /tasks`
- `GET /tasks/{task_id}`
- `PATCH /tasks/{task_id}/status`
- `DELETE /tasks/{task_id}`

2. `app/routers/users.py`

Префикс: `/users`

Маршруты:

- `GET /users/me`
- `GET /users/{user_id}`

3. `app/routers/admin.py`

Префикс: `/admin`

Маршруты:

- `GET /admin/stats`
- `DELETE /admin/tasks/{task_id}`

Требования к зависимостям

Создайте файл `app/dependencies.py` и реализуйте зависимости:

1. `get_current_user`

Считывает заголовки:

```
X-User-Id: 10
X-User-Role: user
```

Возвращает объект пользователя:

```
{
  "id": 10,
  "role": "user"
}
```

Если `X-User-Id` отсутствует или некорректен, верните `401`.

2. `require_admin`

Проверяет, что текущий пользователь имеет роль `admin`.

Если роль не `admin`, верните `403`.

3. `get_storage`

Возвращает объект или словарь, который используется как хранилище задач.

Требования к маршрутам

1. Все маршруты `/tasks` должны использовать `get_current_user`.
2. Все маршруты `/admin` должны использовать `require_admin`.
3. Маршрут `/admin/stats` должен возвращать статистику:

```
{
  "total_tasks": 5,
  "by_status": {
    "todo": 2,
    "in_progress": 1,
    "done": 2
  }
}
```

4. Обычный пользователь не должен иметь доступ к `/admin/stats`.
5. Администратор может удалить любую задачу, даже если он не является её владельцем.

Требования к тестам

Создайте файл `tests/test_dependencies_and_routing.py`.

Проверьте минимум следующие сценарии:

1. `/users/me` возвращает текущего пользователя.
2. Пользователь без заголовка `X-User-Id` получает `401`.
3. Обычный пользователь получает `403` при обращении к `/admin/stats`.
4. Администратор получает статистику по всем задачам.
5. Обычный пользователь не может удалить чужую задачу через `/tasks/{task_id}`.
6. Администратор может удалить чужую задачу через `/admin/tasks/{task_id}`.
7. В Swagger UI маршруты сгруппированы по тегам: `tasks`, `users`, `admin`.

Подсказка: для группировки маршрутов используйте `APIRouter(prefix="/tasks", tags=["tasks"])`.

Не обязательно делать все файлы большими. Главное — чтобы архитектура была понятной, приложение запускалось, а тесты проверяли реальные сценарии взаимодействия компонентов.

Made with ❤️ by **dv0retsky**